# TASK 3 — STEP-BY-STEP GUIDE

## DESIGN OOP MODELS · SWIFT CLASSES, UML & VALIDATION

Course: Geo Programming | Week 5 | Suggested time: 35–50 minutes | Individual work

## 1. Learning Outcomes

Derive domain classes from Task 1 requirements and Task 2 use cases; define properties, Swift types, initializers and methods; model associations and multiplicities; justify encapsulation, inheritance and polymorphism; and specify validation rules.

## 2. Scenario & Inputs

School Management System, academic year 2026–2027: students, teachers, subjects and enrollments. A teacher may teach several subjects; a subject may have several teachers. Students may enroll in several subjects. Each enrollment records an academic year and a mark (0–10). Duplicate enrollment for the same student, subject and academic year is prohibited.

Use your FR/NFR list and use-case table as evidence. Do not begin by coding all classes. Examples below illustrate the METHOD, not the complete design.

## Step 1 — Identify Candidate Domain Classes (5 min)

Underline domain nouns in the specification. Keep persistent business concepts separate from SwiftUI screens and coordinating controllers.

Example 1: Student: a real-world person whose ID, name and email must be recorded.

Example 2: Subject: a course offering described by an identifier, title, credits and semester.

Example 3: StudentListView: NOT a domain class; it displays students and belongs to the UI layer.

| Candidate | Domain / UI / Controller? | Reason |
|---|---|---|
| Student | Domain | Represents a school record |
| StudentListView | UI | Displays records |
| Enrollment | Domain | Records a student's subject, year and mark |
| SchoolController | Controller | Coordinates actions and cross-record rules |
| EnrollmentFormView | UI | Collects enrollment input |

**YOUR TURN — Identify ALL remaining candidate concepts from the specification and use cases. Classify each as Domain, UI or Controller and justify the choice. Continue on extra paper if necessary.**

| Candidate | Layer | Justification |
|---|---|---|
| Teacher | Domain | A real person with ID, name and email who teaches subjects (FR-05, FR-06). |
| Enrollment | Domain | A student taking a subject in an academic year with a mark (FR-07 → FR-10). |
| Teaching assignment | Domain (stored in Teacher) | Many-to-many link; a set of subject IDs inside Teacher is enough, so no extra class. |
| SchoolController | Controller | Owns all lists, coordinates user actions and checks unique IDs and duplicate enrollments. |
| Validator / SchoolError | Domain support | Shared validation rules and readable error messages (FR-12). |

| | | |
|---|---|---|
| Describable / PersonRecord | Abstraction (protocol) | Common display interface for Student, Teacher and Subject; enables polymorphism. |
| StudentFormView, EnrollmentFormView, MarkEditView | UI | Only collect input and show errors; not domain entities. |
| Staff, School | Excluded | Staff is the app user (no login in scope); School is the system boundary, not a record. |

## Step 2 — Assign Responsibilities (4 min)

For each candidate class, write ONE primary responsibility. Do not place all operations in one giant class.

Example 1: Student — represent student identity and contact details.

Example 2: Subject — represent subject identity and academic attributes.

Example 3: Enrollment — represent a student taking a subject in a specific academic year, with a mark.

**YOUR TURN — Determine responsibilities for ALL remaining justified domain classes or supporting abstractions. Avoid unnecessary classes; explain any excluded candidate.**

| Class / abstraction | Primary responsibility | Evidence (FR / UC) |
|---|---|---|
| Student | Represent student identity and contact details; validate its own name and email. | FR-01, FR-03, FR-04 / UC-01, UC-05, UC-06 |
| Teacher | Represent a teacher and the subjects he or she teaches; block duplicate assignment. | FR-05, FR-06 / UC-07, UC-09 |
| Subject | Represent subject identity and academic attributes (credits, semester). | FR-05 / UC-09 |
| Enrollment | Represent one student–subject–year record and protect the mark range. | FR-07, FR-09, FR-10 / UC-02, UC-03 |
| SchoolController | Coordinate add/edit/search/assign/enroll/mark actions; enforce unique IDs and no duplicate enrollment. | FR-02, FR-08, FR-11, FR-12 / all UCs |
| Validator | Check required text, email format, mark 0–10 and academic-year format. | FR-10, FR-12 / error flows |
| Describable / PersonRecord | Shared display interface used by one reusable row view. | FR-01, FR-05 |

## Step 3 — Define Swift Properties, Types & Initializers (6 min)

For each domain class, identify its data, choose Swift types, define access control, and decide which fields are required when creating an instance.

Example 1: Student: id: String, name: String, email: String; init(id:name:email:).

Example 2: Subject: id: String, title: String, credits: Int, semester: Int.

Example 3: Enrollment: academicYear: String, mark: Double; also needs references/IDs to its student and subject.

| Class | Properties + Swift types | Initializer parameters |
|---|---|---|
| Student | id: String; name: String; email: String | id, name, email |
| Subject | id: String; title: String; credits: Int; semester: Int | id, title, credits, semester (all required) |
| Teacher | id: String; name: String; email: String; subjectIDs: Set<String> | id, name, email |
| Enrollment | id: UUID; studentID: String; subjectID: String; academicYear: String; mark: Double? | studentID, subjectID, academicYear (mark optional) |
| SchoolController | students, teachers, subjects, enrollments (arrays) | optional sample data |

**YOUR TURN — Specify ALL remaining classes, properties, Swift types, access levels and initializer parameters. Mark required and optional values. Add rows as needed.**

| Class | Properties / Swift types / access | Initializer |
|---|---|---|
| Student | let id: String; private(set) var name: String; private(set) var email: String — all required | init(id:name:email:) throws |
| Teacher | let id: String; private(set) var name: String; private(set) var email: String; private(set) var subjectIDs: Set<String> = [] | init(id:name:email:) throws |
| Subject | let id: String; private(set) var title: String; private(set) var credits: Int; private(set) var semester: Int | init(id:title:credits:semester:) throws |
| Enrollment | let id: UUID; let studentID, subjectID, academicYear: String; private(set) var mark: Double? (nil = not graded yet) | init(studentID:subjectID:academicYear:mark: Double? = nil) throws |
| SchoolController | private(set) var students: [Student], teachers: [Teacher], subjects: [Subject], enrollments: [Enrollment]; let currentAcademicYear = "2026-2027" | init(students:teachers:subjects:enrollments:) |
| SchoolError (enum) | emptyField, invalidEmail, invalidMark, invalidCredits, invalidSemester, invalidAcademicYear, duplicateID, duplicateEnrollment, duplicateAssignment, notFound | — (enum cases with readable messages) |

## Step 4 — Identify Meaningful Methods & Validation (5 min)

Use verbs from use cases. A method should perform domain behavior or protect an invariant; do not add trivial getters/setters without a reason.

Example 1: Student.updateEmail(_:) — accept a nonempty, valid-looking email or report an error.

Example 2: Enrollment.updateMark(_:) — accept only a score in 0...10.

Example 3: Subject.updateCredits(_:) — reject nonpositive credit values (proposed design rule; label as an assumption).

| Class | Method signature (proposed) | Validation / result |
|---|---|---|
| Enrollment | func updateMark(_ value: Double) throws | 0...10 or error |
| Student | func updateEmail(_ value: String) throws | valid email or error |
| Teacher | func assign(subjectID: String) throws | already assigned → duplicateAssignment error |
| SchoolController | func enroll(studentID:subjectID:academicYear:) throws | duplicate (student, subject, year) → error |
| Subject | func updateCredits(_ value: Int) throws | credits ≤ 0 → error (assumption) |

**YOUR TURN — Derive ALL remaining meaningful methods and validation rules from your use cases. Explain expected outcomes and label proposed rules as assumptions.**

| Class | Method / signature | Validation / expected result |
|---|---|---|
| Student | func update(name: String, email: String) throws | Name not empty, valid email; otherwise error and values unchanged. |
| Teacher | func assign(subjectID:) throws / func unassign(subjectID:) | Duplicate assignment → error; unassign removes the subject. |
| Enrollment | func updateMark(_ value: Double) throws | Accepts 0...10 inclusive; otherwise error and the old mark is kept. |
| SchoolController | func addStudent(id:name:email:) throws | Valid fields and unique ID; otherwise duplicateID error. |
| SchoolController | func enroll(studentID:subjectID:academicYear:) throws | Student and subject must exist; year "YYYY-YYYY"; no duplicate (student, subject, year). |
| SchoolController | func updateMark(enrollmentID:markText:) throws | Empty or non-numeric text rejected; range checked by Enrollment. |
| SchoolController | func averageMark(of student: Student) -> Double? | Average of graded marks; nil when there are no marks. |
| Subject | func updateCredits(_ value: Int) throws | Credits > 0 and semester 1–3 (proposed rules — assumptions). |

## Step 5 — Draw UML Associations & Multiplicities (5 min)

Represent relationships with lines and multiplicities (1, 0..*, etc.). Do not confuse inheritance with association.

Example 1: Student (1) — (0..*) Enrollment: one student can have many enrollment records.

Example 2: Subject (1) — (0..*) Enrollment: one subject can appear in many enrollments.

Example 3: Teacher (0..*) — (0..*) Subject: teaching assignments are many-to-many.

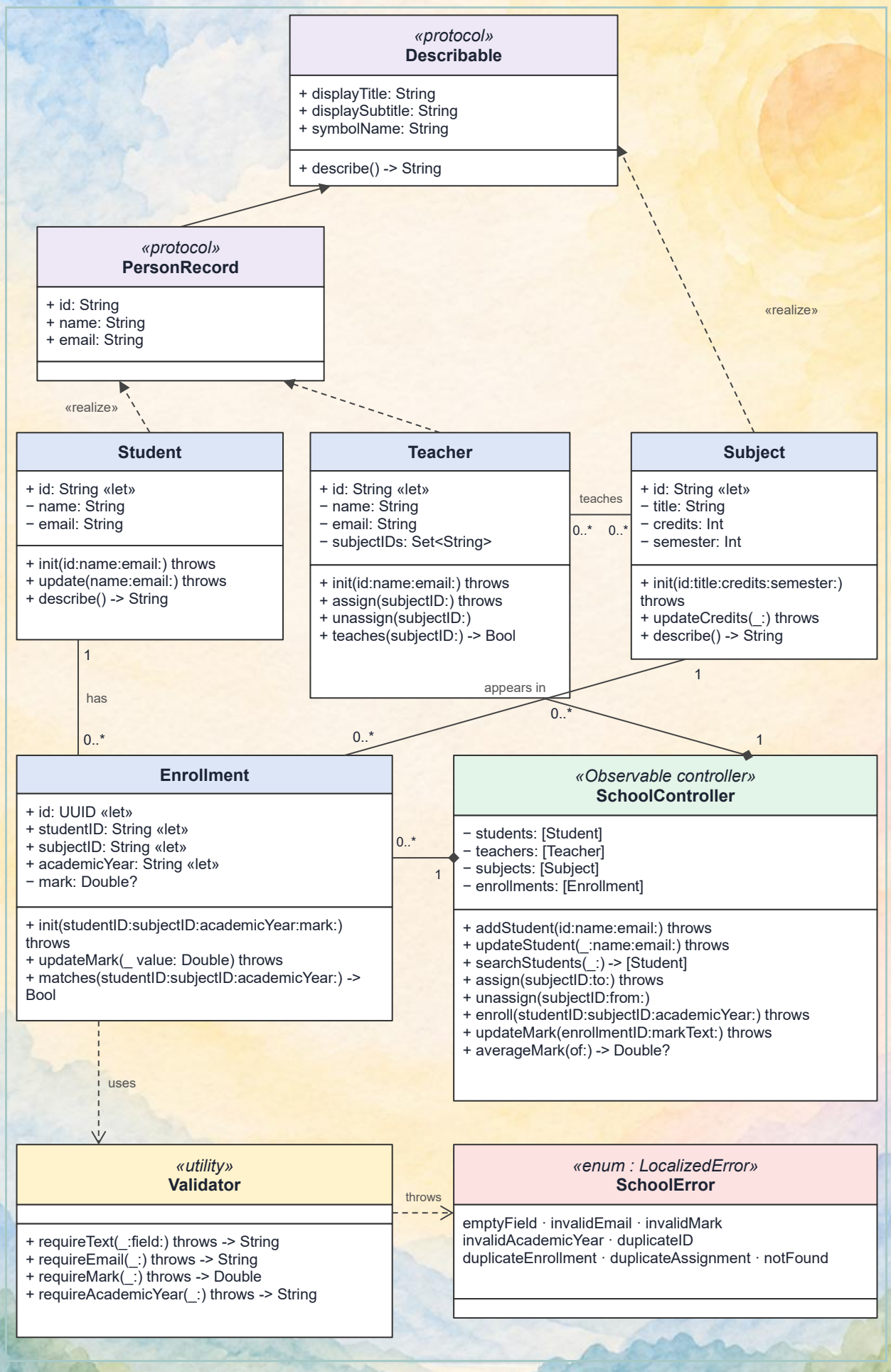
Simple UML-style notation: [Student] 1 ——— 0..* [Enrollment] 0..* ——— 1 [Subject]

| Class A | Multiplicity | Class B | Explanation |
|---|---|---|---|
| Student | 1 → 0..* | Enrollment | Student has enrollments |
| Teacher | 0..* ↔ 0..* | Subject | Teachers teach subjects |
| Subject | 1 → 0..* | Enrollment | A subject appears in many enrollments |
| SchoolController | 1 ◇→ 0..* | Student | The controller holds the student list |

**YOUR TURN — Identify ALL justified associations and multiplicities. Complete the relationship worksheet and draw your full UML-style class diagram in the drawing area.**

| Class A | Multiplicity | Class B | Rationale |
|---|---|---|---|
| Student | 1 — 0..* | Enrollment | One student has many enrollments; each enrollment belongs to one student. |
| Subject | 1 — 0..* | Enrollment | One subject appears in many enrollments; each enrollment is for one subject. |
| Teacher | 0..* — 0..* | Subject | Many-to-many teaching assignment, stored as Teacher.subjectIDs. |
| SchoolController | 1 ◇— 0..* | Student / Teacher | The controller owns the in-memory lists (aggregation). |
| SchoolController | 1 ◇— 0..* | Subject / Enrollment | One shared store for the whole app. |
| Student, Teacher | realizes | PersonRecord (protocol) | Shared identity interface instead of a Person superclass. |
| Subject | realizes | Describable (protocol) | Can be displayed by the same row view as people. |

**UML CLASS DIAGRAM — Draw your complete model here (or attach a separate diagram):**


«protocol»
Describable
+ displayTitle: String
+ displaySubtitle: String
+ symbolName: String
+ describe() -> String
«protocol»
PersonRecord
+ id: String
+ name: String
+ email: String
«realize»
«realize»
Student
+ id: String «let»
− name: String
− email: String
+ init(id:name:email:) throws
+ update(name:email:) throws
+ describe() -> String
Teacher
+ id: String «let»
− name: String
− email: String
− subjectIDs: Set<String>
+ init(id:name:email:) throws
+ assign(subjectID:) throws
+ unassign(subjectID:)
+ teaches(subjectID:) -> Bool
teaches
0..*
0..*
Subject
+ id: String «let»
− title: String
− credits: Int
− semester: Int
+ init(id:title:credits:semester:) throws
+ updateCredits(_:) throws
+ describe() -> String
1
has
0..*
appears in
0..*
0..*
1
1
Enrollment
+ id: UUID «let»
+ studentID: String «let»
+ subjectID: String «let»
+ academicYear: String «let»
− mark: Double?
+ init(studentID:subjectID:academicYear:mark:) throws
+ updateMark(_ value: Double) throws
+ matches(studentID:subjectID:academicYear:) -> Bool
0..*
1
«Observable controller»
SchoolController
− students: [Student]
− teachers: [Teacher]
− subjects: [Subject]
− enrollments: [Enrollment]
+ addStudent(id:name:email:) throws
+ updateStudent(_:name:email:) throws
+ searchStudents(_:) -> [Student]
+ assign(subjectID:to:) throws
+ unassign(subjectID:from:)
+ enroll(studentID:subjectID:academicYear:) throws
+ updateMark(enrollmentID:markText:) throws
+ averageMark(of:) -> Double?
uses
«utility»
Validator
+ requireText(_:field:) throws -> String
+ requireEmail(_:) throws -> String
+ requireMark(_:) throws -> Double
+ requireAcademicYear(_:) throws -> String
throws
«enum : LocalizedError»
SchoolError
emptyField · invalidEmail · invalidMark
invalidAcademicYear · duplicateID
duplicateEnrollment · duplicateAssignment · notFound

## Step 6 — Explain OOP Principles (7 min)

Show HOW the design applies a principle. Inheritance and polymorphism must be justified by real behavior; a protocol or composition may be a better choice.

Example 1: Encapsulation: private(set) var mark: Double; a validated updateMark(_:) method prevents uncontrolled writes.

Example 2: Inheritance (optional): a Person superclass could share name/email between Student and Teacher, but only if common identity/behavior justifies it.

Example 3: Polymorphism: a Describable protocol with describe() can be implemented differently by Student and Teacher; call it through a Describable reference.

Swift starter syntax (illustrative only):

```
protocol Describable { func describe() -> String }
class ExampleEntity {
    private(set) var id: String
    init(id: String) { self.id = id }
    func validate() -> Bool { !id.isEmpty }
}
```

**YOUR TURN — Examine ALL relevant OOP decisions: encapsulation, abstraction, inheritance, overriding, protocols, polymorphism and composition. Explain where each applies or why it is unnecessary.**

| OOP principle / class | Design decision / Swift feature | Why appropriate? |
|---|---|---|
| Encapsulation | private(set) var mark / name / email / subjectIDs; changes only through updateMark, update, assign and enroll. | Views cannot write invalid data directly, so invalid marks or duplicates never silently change data. |
| Abstraction | Describable protocol (displayTitle, displaySubtitle, describe()); Validator utility. | Views work with “something displayable”; rules live in one place. |
| Inheritance (decision) | No Person superclass; Student and Teacher conform to the PersonRecord protocol. | They share data, not behaviour; a protocol avoids a fragile class hierarchy. |
| Overriding | Not used (classes are final); each type gives its own describe() through the protocol. | Same flexibility as overriding without subclassing. |
| Polymorphism | RecordRow(record: any Describable) displays Student, Teacher and Subject; describe() differs per type. | One reusable row view for three different entities. |
| Composition | Teacher holds subjectIDs; Enrollment holds studentID and subjectID; the controller composes all lists. | Models the many-to-many link and ownership without inheritance. |

## Step 7 — Protect Business Rules (5 min)

Turn each business rule into a testable invariant. Decide which model or service checks it, and how errors are reported to the controller/UI.

Example 1: Invalid mark: updateMark(11) must fail and preserve the previous mark.

Example 2: Duplicate enrollment: adding the same student + subject + academic year twice must be rejected.

Example 3: Valid boundary: marks 0 and 10 must be accepted.

| Rule / case | Owner of validation | Expected behavior |
|---|---|---|
| mark = 11 | Enrollment / enrollment service | Reject; keep previous value |
| same student + subject + year | Enrollment manager / repository | Reject duplicate |
| mark = 0 and mark = 10 | Enrollment.updateMark / Validator | Accept (boundary values) |
| mark = −1 | Validator.requireMark | Reject; keep previous value |
| existing student ID (S001) | SchoolController.addStudent | Reject "ID S001 already exists." |

**YOUR TURN — Find ALL meaningful valid, invalid, duplicate and boundary cases implied by the specification. State who enforces each rule and what happens when validation fails.**

| Rule / test case | Validation owner | Expected behavior |
|---|---|---|
| mark = 10.5 or "abc" | SchoolController.updateMark / Enrollment | Reject; previous mark kept. |
| same student + subject, different year (2027-2028) | SchoolController.enroll | Accept — the rule only forbids the same academic year. |
| academic year "2026" or "2026-2028" | Validator.requireAcademicYear | Reject: must be consecutive years YYYY-YYYY. |
| empty name or empty ID | Student.init / Student.update | Reject "... is required."; nothing stored. |
| invalid email "kiet@" | Validator.requireEmail | Reject; record unchanged. |
| same subject assigned twice to one teacher | Teacher.assign | Reject duplicate assignment. |
| enroll an unknown student or subject | SchoolController.enroll | Reject "... was not found." |

## Step 8 — Review, Trace & Submit (3 min)

Trace your classes and operations back to Task 1 FRs and Task 2 use cases.

Example 1: Student + student listing → FR for browsing students.

Example 2: Enrollment.updateMark → use case for entering/updating marks.

Example 3: Duplicate-enrollment check → business rule and alternate/error flow.

| Requirement / use case | Model or method | Evidence / test |
|---|---|---|
| FR-01 | SchoolController.students + StudentListView | Student list shows ID and name |
| UC-03 | Enrollment.updateMark | 8.5 saved; 11 rejected |
| FR-08 | SchoolController.enroll (duplicate check) | Second identical enrollment rejected |
| UC-05 | SchoolController.addStudent | New student appears in the list |

**YOUR TURN — Trace ALL remaining classes, methods and validation decisions to Task 1 requirements and Task 2 use cases. Identify any uncovered requirement.**

| FR / UC ID | Model / method / rule | Evidence / test |
|---|---|---|
| FR-02 / UC-04 | SchoolController.searchStudents(_:) | Search "S003" → one result |
| FR-04 / UC-06 | Student.update(name:email:) | Edited email shown in detail |

| | | |
|---|---|---|
| FR-05 / UC-09 | SchoolController.subjects, teachers(of:) | Subject detail lists its teachers |
| FR-06 / UC-07 | Teacher.assign / unassign | MAT101 appears under T01; duplicate rejected |
| FR-07 / UC-02 | SchoolController.enroll + Enrollment.matches | New enrollment listed |
| FR-10 / UC-03 (invalid mark) | Validator.requireMark | Mark 11 rejected; old mark kept |
| FR-11 / UC-08 | SchoolController.enrollments(of:), averageMark(of:) | Student detail shows marks and average |
| FR-12 / UC-05 (invalid data) | Validator.requireText / requireEmail, duplicateID | Empty name, bad email, S001 rejected |
| NFR-01 | SampleData: 25 students × 4 subjects = 100 enrollments | Lists interactive within 2 s |
| Uncovered requirements | None — every FR-01 → FR-12 maps to a model or method | — |

## 3. Required Student Deliverables

☑ Candidate-class classification with responsibilities and rationale.

☑ Completed model design worksheet: attributes/types, access control, initializers, methods and validation.

☑ One UML-style class diagram showing ALL justified associations and multiplicities.

☑ OOP explanation: encapsulation plus a justified inheritance/protocol/composition decision; show polymorphism if appropriate.

☑ Business-rule table with duplicate-enrollment and mark-range checks, including boundary cases.

☑ Traceability table linking model decisions to Task 1 FRs and Task 2 use cases.

Save as Task3_OOP_Models_StudentID.pdf and include it in Design.pdf for Lab 01. Submit your own design; the worked examples are not a complete solution.

## 4. Self-Assessment Checklist

☑ Every domain class has one clear responsibility.

☑ Swift types and initializers are specified for every class.

☑ UI views and controllers are not mistaken for domain entities.

☑ UML relationships have correct multiplicities.

☑ No unnecessary inheritance hierarchy has been forced.

☑ Invalid marks and duplicate enrollments cannot silently alter data.

☑ Every key model behavior traces to a requirement or use case.

## 5. Common Mistakes

| Common mistake | Better approach |
|---|---|
| Put all data and logic in ContentView | Separate domain models, views and coordination |
| Use inheritance only because fields repeat | Prefer composition/protocols unless subtype behavior is meaningful |
| Expose mark as freely writable | Protect it with validated mutation |
| Draw UML with no multiplicities | Add 1, 0..*, etc., to every association |
| Invent new business rules as facts | Label them as design assumptions |

Reflection (3–5 sentences): Which relationship or validation rule most influenced your model design? What trade-off did you make between inheritance and composition?

The duplicate-enrollment rule influenced my design the most: it involves three values (student, subject, academic year) and the whole list of enrollments, so it cannot be checked inside one Enrollment object and is enforced by SchoolController. Rules that belong to a single object, such as the mark range, stay inside Enrollment through private(set) and a validated updateMark(_:) method. Between inheritance and composition, I chose protocols and composition: Student and Teacher share only identity fields, not behaviour, so a Person superclass would add a hierarchy without real benefit. The PersonRecord and Describable protocols still let one view display all records polymorphically.